

Project Report: CrisisRoute Logistics System

Team Members: Tao Geng, Keval Patel

1. Description of the Project

The objective of this project was to develop **CrisisRoute**, an algorithm-centered web application designed to support decision-making for crisis response in active conflict zones.

2. Significance of the Project

In modern warfare and disaster scenarios, standard GPS routing fails because road safety and infrastructure viability fluctuate wildly. This project is highly meaningful because it tackles the uncertainty of these environments using computational complexity theory. The novelty of CrisisRoute lies in its departure from static pathfinding. Rather than simply returning the shortest path via greedy algorithms, it layers a **Monte Carlo Statistical Simulation** over graph traversal to account for sudden ambushes, weather, or road closures, outputting a "Probability of Survival." Furthermore, upgrading standard resource allocation to a 2D dynamic programming model ensures that life-saving cargo calculations account for the physical reality of truck beds, making the tool practically applicable to real-world humanitarian supply chains.

3. Code Structure

The project utilizes a multi-tier architecture, separating complex Python algorithms from the interactive JavaScript frontend, with SQLite providing data persistence.

Plaintext

CrisisRoute_Project/

```
|
├── app.py          # Backend Flask Server & Core API Routes
│   ├── optimize_supplies_2d() # 2D Knapsack DP logic
│   ├── get_route_and_sim()    # Dijkstra + Monte Carlo logic
│   └── init_db()              # SQLite database initialization
├── crisis_logs.db      # Auto-generated SQLite database (Stores history)
├── requirements.txt    # Dependencies (Flask, gunicorn, numpy)
├── templates/
│   ├── index.html      # Frontend User Interface
│   ├── vis-network.js  # Procedural DAG map generation & interaction
│   ├── chart.js        # Payload and Risk analytics visualizations
│   └── fetch/API       # Asynchronous calls to Flask backend
```

- **Backend (app.py):** Handles all heavy computational lifting and database reads/writes.

- **Frontend (index.html):** Maintains the application state, handles procedural graph generation, and dynamically updates UI components without page reloads.

4. Description of Algorithms

Dijkstra's Algorithm + Monte Carlo Simulation

We first run Dijkstra's Algorithm using a priority queue to have a baseline shortest path through the generated Directed Acyclic Graph (DAG). Because edge weights (danger levels) are volatile in war zones, we pass this baseline path into a Monte Carlo simulator. The simulation runs 1,000 iterations, applying a randomized multiplier to every edge on the path. This calculates an average simulated risk and the statistical percentage of iterations that fall within an acceptable survival threshold.

5. Verification of Algorithms

2D Knapsack Verification:

- *Constraint:* Weight: 50, Volume: 40
- *Items:* Trauma Kits (W:20, V:15, Val:100), Water (W:30, V:20, Val:80), Thermal Blankets (W:10, V:25, Val:50).
- *Trace:* A standard 1D knapsack based only on weight would select Trauma Kits + Water (Weight 50, Value 180). However, the combined volume of those items is 35. If we lower the volume cap to 30 for testing, the 2D algorithm correctly rejects the combination and selects Trauma Kits + Blankets (Weight 30, Vol 40, Value 150), proving the dual-constraint bounds hold.

Monte Carlo Verification:

- *Trace:* Given a static path Basecamp -> Zone D -> Refugee Camp with base edge weights of 10 and 12, the baseline risk is 22. During the 1,000-loop execution, the algorithm successfully aggregates the randomized iterations, frequently resulting in an average simulated risk of ~36.5, proving the uniform distribution properly scales the baseline danger.

6. Execution Results and Analysis

The system successfully integrates algorithmic processing with a highly interactive user experience.

- **Procedural Map Generation:** Our JavaScript dynamically generates a new Directed Acyclic Graph (DAG) with 5 to 8 nodes and randomized edge weights upon request. This forces the algorithms to process novel data structures continuously.
- **Visualizations:** Vis.js accurately highlights the optimal route `_plan.path` array in red against the gray network map. Chart.js successfully renders a doughnut chart of the loaded supply composition and a bar chart contrasting the baseline risk against the Monte Carlo simulated risk.
- **Data Persistence:** Executions are instantly logged to the SQLite `crisis_logs.db` and rendered dynamically into the frontend history table, demonstrating successful full-stack data flow.

The main result is a system that mathematically proves that optimal logistical dispatching cannot rely purely on shortest-path assumptions when resource constraints are multi-dimensional and environmental risks are highly volatile.

7. Conclusions

We successfully translated complex theoretical concepts—NP-hard approximations, dynamic programming, and statistical modeling—into a practical, user-facing web application.

- **Findings:** Decoupling the knapsack problem from the graph traversal allows both algorithms to run in fractions of a second, ensuring the UI remains highly responsive even when executing 1,000 simulation loops.
- **Project Issues:** Currently, edge weights are simulated via random number generation. A future, production-ready iteration would require integrating live API endpoints (e.g., satellite data or crowdsourced intelligence) to adjust these weights dynamically based on real-time combat data. Additionally, very dense procedural graphs can occasionally clutter the Vis.js canvas.
- **Course Application:** This project served as a comprehensive synthesis of our senior-level coursework, directly applying algorithm design, time-complexity analysis, and secure data handling to a real-world software engineering pipeline.

8. AI Usage

AI tools were utilized strategically throughout the development cycle to accelerate tasks and troubleshoot specific environment issues.

- **Debugging:** AI was used to diagnose and resolve a `jinja2.exceptions.TemplateNotFound` error and a severe `IndentationError` during the Gunicorn deployment phase on the Render hosting platform.
- **Algorithmic Verification:** AI was utilized as a sounding board to verify the mathematical formulation of the 3D array matrix used in the 2D Knapsack DP algorithm, ensuring it handled out-of-bounds indices correctly during state transitions.
- **Used to write parts of this report**